



在 docker 中使用 mihomo

📅 2024年11月02日

🏠 Tech

须知

本文默认您已经安装好了 docker 并了解 docker 的一些基本概念

如果您是纯命令行环境，本文将默认你会命令行最基本的使用（如创建文件/文件夹、编辑文本等）

mihomo 是 clash 的一个分支，也是目前使用最广泛的分支，各平台许多客户端使用的核心均为 mihomo，或者它曾经还有个可能更多人听过的名字叫 **mihomo meta**。

需要说明的是 mihomo 与最原始的 clash 本身都是一个纯命令行程序，这意味着它本身没有图形界面，但本文会讲到如何配置 mihomo 的 **webUI**，以便日常使用。

编写 mihomo 配置文件

新建一个空文件夹命名为 **mihomo**，在其中新建文本文档 **config.yaml**

可参考下方模板来创建您自己的配置文件，模板参考自官方文档，如果你希望了解配置文件的具体规则，同样请参考官方文档。（本文最后有链接）

该模板中**必须要修改**的是第 19、20 行与 25 行，根据实际情况进行填写，修改模板后不含模板中的尖括号。其中节点提供者名称可自由填写，用于程序区分多个订阅；节点名称前缀会在显示节点信息时插入到原始节点名称前，便于自己区分不同订阅的节点。

第 1 行为代理端口设置，第 6 行为外部控制端口设置（如使用 webUI 控制 mihomo）如有需要可自行更改。

```
1 mixed-port: 7890
2 allow-lan: true
3 bind-address: '*'
4 mode: rule
5 log-level: info
6 external-controller: '0.0.0.0:9090'
```



```
7
8 tun:
9   enable: false
10  stack: mixed
11  dns-hijack:
12    - "any:53"
13    - "tcp://any:53"
14  auto-route: true
15  auto-redirect: true
16  auto-detect-interface: true
17
18 proxy-providers:
19   <节点提供者名称>:
20     url: "<节点订阅链接>"
21     type: http
22     interval: 86400
23     health-check: {enable: true,url: "https://www.gstatic.com/gen
24     override:
25       additional-prefix: "<节点名称前缀>"
26
27 proxies:
28   - name: "直连"
29     type: direct
30     udp: true
31
32 geodata-mode: true
33 geox-url:
34   geoip: "https://github.com/MetaCubeX/meta-rules-dat/releases/do
35   geosite: "https://github.com/MetaCubeX/meta-rules-dat/releases/
36   mmdb: "https://github.com/MetaCubeX/meta-rules-dat/releases/dow
37   asn: "https://github.com/MetaCubeX/meta-rules-dat/releases/down
38
39 dns:
40   enable: true
41   ipv6: true
42   respect-rules: true
43   enhanced-mode: fake-ip
44   fake-ip-filter:
45     - "*"
46     - "+.lan"
47     - "+.local"
48     - "+.market.xiaomi.com"
49   nameserver:
50     - https://120.53.53.53/dns-query
51     - https://223.5.5.5/dns-query
52   proxy-server-nameserver:
53     - https://120.53.53.53/dns-query
54     - https://223.5.5.5/dns-query
55   nameserver-policy:
56     "geosite:cn,private":
57       - https://120.53.53.53/dns-query
```

```
58     - https://223.5.5.5/dns-query
59     "geosite:geolocation-!cn":
60     - "https://dns.cloudflare.com/dns-query"
61     - "https://dns.google/dns-query"
62
63 proxy-groups:
64
65     - name: 默认
66       type: select
67       proxies: [自动选择,直连,香港,台湾,日本,新加坡,美国,其它地区,全部节点]
68
69     - name: Google
70       type: select
71       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
72
73     - name: Telegram
74       type: select
75       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
76
77     - name: Twitter
78       type: select
79       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
80
81     - name: 哔哩哔哩
82       type: select
83       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
84
85     - name: 巴哈姆特
86       type: select
87       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
88
89     - name: YouTube
90       type: select
91       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
92
93     - name: NETFLIX
94       type: select
95       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
96
97     - name: Spotify
98       type: select
99       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
100
101     - name: Github
102       type: select
103       proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动选择]
104
105     - name: 国内
106       type: select
107       proxies: [直连,默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点]
108
```

```
109 - name: 其他
110   type: select
111   proxies: [默认,香港,台湾,日本,新加坡,美国,其它地区,全部节点,自动
112
113 #分隔,下面是地区分组
114 - name: 香港
115   type: select
116   include-all: true
117   filter: "(?i)港|hk|hongkong|hong kong"
118
119 - name: 台湾
120   type: select
121   include-all: true
122   filter: "(?i)台|tw|taiwan"
123
124 - name: 日本
125   type: select
126   include-all: true
127   filter: "(?i)日|jp|japan"
128
129 - name: 美国
130   type: select
131   include-all: true
132   filter: "(?i)美|us|unitedstates|united states"
133
134 - name: 新加坡
135   type: select
136   include-all: true
137   filter: "(?i)(新|sg|singapore)"
138
139 - name: 其它地区
140   type: select
141   include-all: true
142   filter: "(?i)^(?!.*(?:HK|JP|US|SG|CN|港|hk|hongkong|台|tw|taiwan|日|jp|japan|美|us|unitedstates|united states|新|sg|singapore))"
143
144 - name: 全部节点
145   type: select
146   include-all: true
147
148 - name: 自动选择
149   type: url-test
150   include-all: true
151   tolerance: 10
152
153 rules:
154 - GEOIP,lan,直连,no-resolve
155 - GEOSITE,github,Github
156 - GEOSITE,twitter,Twitter
157 - GEOSITE,youtube,YouTube
158 - GEOSITE,google,Google
159 - GEOSITE,telegram,Telegram
```

```
160 - GEOSITE,netflix,NETFLIX
161 - GEOSITE,bilibili,哔哩哔哩
162 - GEOSITE,bahamut,巴哈姆特
163 - GEOSITE,spotify,Spotify
164 - GEOSITE,CN,国内
165 - GEOSITE,geolocation-!cn,其他
166
167 - GEOIP,google,Google
168 - GEOIP,netflix,NETFLIX
169 - GEOIP,telegram,Telegram
170 - GEOIP,twitter,Twitter
171 - GEOIP,CN,国内
172 - MATCH,其他
```

使用 docker compose 构建容器

在另一处新建一个文件夹，创建 `docker-compose.yml` 文件，填入以下内容，其中 21 行尖括号内需自行修改为你**上一步创建的配置文件**所在目录，修改后**不含尖括号**。

“

注：在另一处创建文件夹是个人习惯，**如果你希望项目使用的文件都在同一处**可以自行调整该 `compose` 文件。如果又想改又实在不会可以将本文开头到下方文件复制拿去问 AI 怎么改。

”

```
1  version: '3'
2
3  services:
4    metacubexd:
5      container_name: metacubexd
6      image: ghcr.io/metacubex/metacubexd
7      restart: always
8      ports:
9        - '9097:80'
10
11   mihomo:
12     container_name: mihomo
13     image: docker.io/metacubex/mihomo:latest
14     restart: always
15     pid: host
16     ipc: host
17     network_mode: host
18     cap_add:
19       - ALL
20     volumes:
21       - <config.yaml所在目录>:/root/.config/mihomo
22       - /dev/net/tun:/dev/net/tun
```

你或许会注意到 compose 文件中除了指定了 mihomo 的镜像外还指定了一个叫做 metacubexd 的镜像，这个镜像是 mihomo 作者开发的一个 mihomo 的 webUI，如果你不需要 webUI 可以将这部分删除（你不知道自己需不需要那就是需要）。

其中第 9 行的 9097 为webUI 访问端口，如有需要可自行更改。

终端

```
1  # 先 cd 到 docker-compose.yml 所在路径
2  # 启动项目
3  sudo docker-compose up -d
4  # 关闭项目
5  sudo docker-compose stop
```

飞牛OS

Docker → Compose → 新增项目 → 名称随意，选择 docker-compose.yml 所在文件夹作为项目路径 → 选择使用现有的 docker-compose 配置文件来创建项目 → 确定

第一次使用需要先构建该项目，后续无需重复构建，直接启动即可。若更新镜像则仍需重新构建一次。

补充

你有可能会运行容器失败并在容器日志中出现如下报错：

```
1 level=fatal msg="Parse config error: rules[512] [GEOIP,CN,DIRECT] e
```

你也肯定能看出来这是因为无法下载 MMDB 导致的，那么解决方法也很明了了，直接去下载一个就行

可以前往[该仓库](#)下载 geoip.metadb 等文件，并将其上传到与 config.yaml 相同的目录内再重新运行容器即可

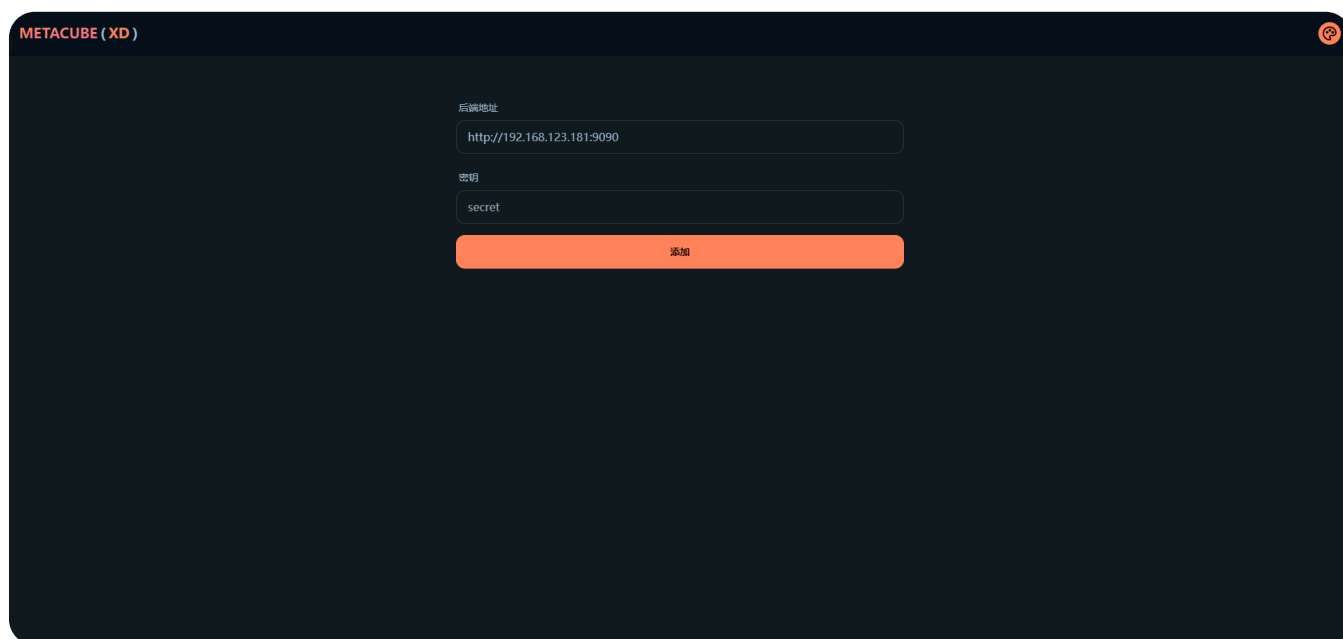
打开 webUI 并开始使用

在局域网内任意设备浏览器中输入 设备IP:9097 即可访问 metacubexd 的界面，后端地址填写 http://设备IP:9090 ，密钥留空即可，如图所示，点击添加便可管理 mihomo 的各项配置。若在前文中修改了 webUI 访问端口和外部控制端口，请自行替换为自己设置的端口号。

“

如果你使用 metacubexd 更改了一些设置，那将只会在 mihomo 的本次运行生效，重启 mihomo 会将所有设置重置为你第一步在 config.yaml 中所填写的配置

”



至此已搭建完成，在需要使用网络代理的地方（具体见 Q&A）在代理选项内填入 设备 IP:7890 即可使用

metacubexd 中一些基本的说明

模板文件的分流规则逻辑

如果你是使用我上方提供的配置模板，那么 metacubexd 的**代理**页面应该有默认、Google、GLOBAL等分组，这些分组被称为**代理组**，你可以指定各个代理组使用哪个节点。

在 metacubexd 的**配置**页面中的运行模式里，分别有规则、直连、全局几个选项。若使用全局，则所有经过 mihomo 的流量均会通过 **GLOBAL代理组** 内指定的节点转发；若使用规则，则会按配置文件中的规则进行分流，下方会介绍模板文件中的规则逻辑；若使用直连就不用我说了吧

我提供的模板文件中的规则为：当访问一个地址时会优先查询是否**属于代理组中已经指定的网站**，比如 Google，这类网站会直接走你在对应代理组中指定的节点。代理组可以套娃，你如果打开 Google 代理组会注意到里面并不让你选择具体的节点，而是选择其他代理组，则这时会再到对应的代理组中使用你指定的节点。

举例：Google 代理组中指定了“香港”这个代理组，而香港代理组中指定了“香港01”这个节点，那么当访问 Google 时，流量会通过“香港01”这个节点转发。

若访问的网站**不属于**代理组中指定的网站，则会使用“默认”代理组中指定的节点，同样可以套娃。

如果想给模板文件中没有指定的网站进行单独分流，请期待后续的进阶教程。先鸽着等我想写再说

TUN 模式

TUN 模式即虚拟网卡模式，启用该模式会另 mihomo 接管本设备的所有流量，如有需要可在 metacubexd 的**配置**页面启用，如果希望启动 mihomo 时自动打开 TUN 模式，可以将模板配置文件中第 9 行的 false 改为 true。

Q&A

Q1：“在需要使用网络代理的地方填入”是指填入哪

A：包括但不限于：各系统的系统代理（如 Windows 设置中的代理设置，手机 WiFi 的代理设置）、Linux 中环境变量的 http_proxy 等代理字段、各软件内部的代理设置等。

- 1 对于 **系统代理**，以 Windows 举例，并非所有软件都会走系统代理，这种时候你需要查看应用内是否能设置代理；若没有相关选项且应用本身不走系统代理则你需要考虑其他方法让 mihomo 来接管流量，不过这些就不在这篇教程的范畴内了。

- 2 如果你希望 mihomo 接管 **docker 宿主设备本身** 的所有流量，你可以参考我的教程启用 **TUN 模式**
- 3 如果你希望作为路由设备使用，比如你可能在其他地方见过的将网关指向该设备的用法，你可以配置 **透明代理**，这同样不在这篇教程的范畴中，但我会推荐你在路由设备中使用 **ShellCrash**（通用）或者 **OpenClash**（仅限 OpenWRT），但详细的使用方法请自行探究。

Q2: 我使用了其他地方寻找的 config.yaml 模板，最后一步点击添加连接后端时报错 Failed to fetch

A: 检查 config.yaml 内的 external-controller 字段，连接后端时以填入该处指定的端口

Q3: 我使用了你之前的文章的 config.yaml 模板，依然出现上一个问题

A: 检查是否将 mihomo 容器的网络模式更改为 host，若没有请去更改

Q4: metacubexd 和 yacd 都有在线版本，为什么不用？

A: 因为我发现在线版本我连接不上 NAS 的后端，原因未知，有人知道原因的话麻烦评论留言告诉我一下。

Q5: 为什么 WebUI 选择了 metacubexd 而不是 yacd 或其他？

A: 个人选择，你当然可以用 yacd，使用上不会有什么影响。

还有其他问题的欢迎留言，如果有其他比较典型的问题我会在后续添加到 Q&A 部分

相关链接

mihomo 仓库: <https://github.com/MetaCubeX/mihomo>

mihomo 文档: <https://wiki.metacubex.one/>

metacubexd 仓库: <https://github.com/MetaCubeX/metacubexd>

本文作者: windowBR

本文链接: <http://blog.windowbr.top/2024/11/02/mihomo-docker/>

版权声明: 本文采用 CC BY-NC-SA 3.0 CN 协议进行许可

Error: Network Error



说点什么

 支持 Markdown 语法

使用 GitHub 登录

预览

来做第一个留言的人吧!